

Efficient Modular Multiplication Using Vector Instructions on Commodity Hardware

Simon Langowski
MIT CSAIL
slangows@mit.edu

Srinivas Devadas
MIT CSAIL
devadas@mit.edu

Abstract—Modular arithmetic is the computational backbone of many cryptographic and scientific algorithms. In particular, modular multiplication in a large prime field is computationally expensive and dictates the runtime of many algorithms. While it is relatively easy to utilize vectorization to accelerate batches of independent modular multiplications, our goal is to reduce the latency of a *single* modular multiplication under a generic prime using vectorization, while maintaining constant-time execution.

We investigate the technique of Residue Number System (RNS) Montgomery modular multiplication. We first contribute a unified view of algorithmic optimizations in prior art. This view enables us to further reduce the number of elementwise multiplications in an algorithm with a simplified structure that we prove correct.

We explore AVX512 acceleration on CPUs, and show how to map our algorithm to vector instructions. We implement our algorithm in C++ and achieve $\approx 4\times$ speedup, which is nearly maximal, for 1024+-bit primes on a CPU with AVX512 over optimized library implementations of standard Montgomery modular multiplication algorithms. GPUs contain vector cores that each support tens of physical threads. We show how to intelligently map our algorithm to threads in a vector core, “overparallelizing” to minimize latency. We show substantial speedups over a commercial library implementation of standard modular multiplication algorithms across a wide range of prime sizes.

Index Terms—modular multiplication, residue number system, vector instructions, Montgomery multiplication

1. Introduction

Modular multiplication plays a crucial role in modern cryptography, forming the backbone of many encryption algorithms. At its core, modular multiplication ensures that operations are performed within a finite number set, which is a key requirement for cryptographic systems to maintain efficiency and security. For example, in public-key cryptographic schemes such as RSA, and those based on elliptic curves, modular exponentiation is fundamental to both the encryption and decryption processes – it repeatedly uses modular multiplication. Diffie-Hellman key exchange also uses modular exponentiation. Furthermore, advanced cryptographic protocols that construct Zero-Knowledge Proofs or

perform Fully Homomorphic Encryption comprise billions of modular multiplication operations for each invocation. Efficient and secure implementations of modular multiplication are therefore critical for performance in real-world systems. In short, modular multiplication is a foundational component of digital security.

Modular addition is easy, especially if the operands are already field elements, however, modular multiplication can be very expensive. For some composite moduli p , modular operations can be decomposed using the Residue Number System (RNS). This occurs only when p factors into the product of small moduli, typically efficient for the word (e.g., $w = 32$ bit) size of the machine. However, in many cryptographic schemes, we would like to have large prime (or otherwise not RNS-friendly) moduli p . For these primes, multiplication and modular reduction dominate the runtime. There are two main algorithms for modular reduction: Barrett’s algorithm and Montgomery’s algorithm, which, for general $t * w$ -bit moduli, run with complexity $\Theta(t^2)$ single-precision multiplications.

One technique to speed up a single modular multiplication is to use RNS-based Montgomery multiplication [1] and exploit parallelism in the hardware platform. The idea is that if there are t moduli, operations on each modulus can be carried out in parallel, reducing overall latency. RNS Montgomery multiplication has a fairly long history of proposed use in ASICs (i.e., accelerators built for a specific purpose) for cryptography. However, ASICs are expensive to design and build, and most proposals have not been implemented on silicon, let alone in commercial silicon. There is also extensive work on using Field Programmable Gate Arrays (FPGAs) to speed up RNS Montgomery multiplication; the issue here is that the circuits on the FPGAs are usually clocked 10-50 $\times$ slower than commercial CPUs, and so even with potentially unbounded parallelism, it is difficult to beat optimized CPU implementations of standard Montgomery multiplication. Also, note that if we have a batch of independent (or unrelated) modular exponentiations (or multiplications) to perform, e.g., to sign several documents, it is easy to exploit this parallelism on multicore CPUs, on vector processors, or GPUs. Not surprisingly, to our knowledge, libraries corresponding to modular arithmetic (e.g., Flint, Arkworks) do not include RNS Montgomery multiplication, and neither does Nvidia’s CGBN modular arithmetic library.

We aim to change this narrative for RNS Montgomery multiplication. In this paper, we explore the applications of RNS-based Montgomery multiplication on commodity hardware, in particular, CPUs with vector instructions (e.g., AVX512 available on the Intel IceLake core¹), as well as GPUs, which have tens of vector cores, each capable of supporting 128 threads. While there has been some works (e.g., [2] for AVX, and [3] for GPUs) in this direction, these works have not resulted in appreciable enough speedups for a *single* modular multiply, in comparison to optimized library implementations, to be attractive alternatives, except, perhaps, for primes of huge sizes. We show that proper exploitation of vectorization through algorithmic optimization (cf. Section 3) and careful mapping to CPU AVX instructions (cf. Section 4.4) and threads in a GPU vector core (cf. Section 4.5) can, in fact, reduce the size of a prime required for vectorization to be effective on commodity hardware from huge sizes to those commonly used in cryptography.

Certain kinds of primes such as Mersenne primes of the form $2^n - 1$ and pseudo-Mersenne primes of the form $2^n - w$ for small w simplify modular reduction. For example, Curve25519 uses $p = 2^{255} - 19$. However, many cryptographic protocols, e.g., RSA, require random primes. Our focus in this paper is to speed up modular reduction for *general* primes, and we do not preclude protocol-level optimizations that use special properties of primes. It is also important to note that all the algorithms described and their implementations on CPUs and GPUs are *constant time*, and do not depend on the specific numbers.

We summarize our contributions below:

- 1) In order to provide a unified view of prior work (cf. Table 1), we introduce pre- and post- multiplication (cf. Section 2.5) into the Change RNS modulus (CRNS) step of the Posch and Posch [1] framework.
- 2) This unified view leads us to a new algorithm (cf. Section 3) with fewer multiplications and a simplified structure that integrates several optimizations, and which we conjecture has no unnecessary elementwise modular multiplications. We argue correctness of our algorithm. Our transformations are such that we can vectorize virtually the entire new algorithm (cf. analysis of Section 5.3).
- 3) We show how to intelligently map our algorithm to a CPU with AVX512 instructions, contributing several optimizations. We outperform a state-of-the-art CPU library for modular arithmetic, Flint, on AVX512 CPUs by a near-optimal $\approx 4\times$ for 1024+-bit primes, and are faster for primes over 512 bits.
- 4) We show how to (over)parallelize our algorithm on a single GPU vector core using as many threads as are available, nearly independently from the size of the prime. Overparallelization not only has performance benefits, but also has the added benefit of being able to use the entire memory of a vector

1. These instructions were limited to server processors only in later generations.

core, which is important for large primes. Our implementation outperforms Nvidia’s CGBN library by $2\times$ or more for primes greater than 512 bits.

- 5) We (will) contribute open-source implementations for both CPUs and GPUs with a convenient API to the community, which we hope will be widely used. Our code is available for review here ².

The paper is organized as follows. We provide background, and describe prior algorithms for RNS Montgomery multiplication in depth in Section 2. Our optimized algorithm for RNS Montgomery multiplication, and a proof of correctness are provided in Section 3. Mapping to parallel commodity hardware is the subject of Section 4. We provide analytical performance comparisons in Section 5 that show that the implementation of our algorithm is almost fully vectorized. We provide extensive experimental results in Section 6, where we compare the performance of our algorithm’s implementations on CPUs with AVX512 and GPUs against state-of-the-art libraries, and show substantial speedups. Additional related work, not covered in Section 2, is summarized in Section 7. We conclude the paper in Section 8. Extensions to several algorithms that are helpful for implementing other functions used by cryptographic applications are described in Appendix D.

2. Background and Prior Algorithms

Modular multiplication is the combined operation of multiplication and modular reduction. There are two inputs, a and b , which have an unreduced product $s = ab$. Modular reduction modulo p (represented as $\%p$) finds the number r in the range $[0, p)$ such that $r + qp = s$. A standard modular multiplication algorithm is shown in the first row of Table 1:

- 1) Multiply a and b to compute the unreduced product s .
- 2) Find the quotient q of s with the modulus p .
- 3) To find the remainder we compute $s - q * p$.

However, on commodity hardware the above algorithm is inefficient, particularly the division required in step 2. There are two standard techniques to remove this division, called Barrett’s algorithm [7] and Montgomery’s algorithm [4], the second and third rows, respectively, in Table 1, which precompute values based on the modulus p in order to remove expensive divisions by p . Instead of dividing by p , these algorithms divide by a number 2^w , e.g., $w = 32$ or $w = 64$ depending on the hardware, as divisions by a power of 2 can be implemented efficiently via bitshifting. The RNS Montgomery algorithm will be based off of the Montgomery algorithm, so we describe it in more detail.

2.1. Montgomery modular multiplication

Modular arithmetic induces a mathematical structure called a ring (or a finite field when p is prime), and there are

2. <https://github.com/SimonLangowski/RNSMont>

Alg.	Representation	Product	Quotient	Remainder	Notes
<i>Basic</i>	$a' = a$	$s = a'b'$	$q = \lfloor \frac{s}{p} \rfloor$	$r = s - qp$	Inefficient
<i>Barrett</i>	$a' = a$	$s = a'b'$	$q = \lfloor \frac{sf}{2^w} \rfloor$	$r = s - qp$	$f = \lfloor \frac{2^w}{p} \rfloor$ $w \geq \lceil \log_2(p) \rceil$
<i>Mont simple</i>	$a' = (a2^w)\%p$	$s = a'b'$	$q = (sd')\%2^w$	$r = \frac{s-qp}{2^w}$	$d' = p^{-1}\%2^w$ $w \geq \lceil \log_2(p) \rceil$
<i>Mont [4]</i>	$a' = (aM)\%p$	$s = a'b'$	$q = (sd)\%M$	$r = \frac{s+qp}{M}$	$d = -p^{-1}\%M$
<i>Modular Mont</i>	$a' = (aM)\%p$	$s = a'b'$	$q = (sd)\%M$	$r = (s + qp)(W)\%N$	$d = -p^{-1}\%M$ $W = M^{-1}\%N$
<i>RNS Mont [1]</i>	$a'_i = ((aM)\%p)\%m_i$ $a'_j = ((aM)\%p)\%n_j$	$s_i = a'_i b'_i \%$ $\%m_i$ $s_j = a'_j b'_j \%$ $\%n_j$	$q_i = (s_i d_i)\%m_i$ $q_j = \text{CRNS}_{N*1}^{M*1}(q_i)$	$r_j = (s_j + q_j p_j)(W_j)\%n_j$ $r_i = \text{CRNS}_{M*1}^{N*1}(r_j)$	$d_i = -p^{-1}\%m_i$ $W_j = M^{-1}\%n_j$ $p_j = p\%n_j$
<i>D-RNS simple</i>	$a'_i = (((aM)\%p)2^w)\%m_i$ $a'_j = (((aM)\%p)2^w)\%n_j$	same as above w/%	$q_i = (s_i d_i)\%m_i$ $q_j = \text{CRNS}_{N*2^w}^{M*2^w}(q_i)$	$r_j = (s_j + q_j p_j)(W_j)\%n_j$ $r_i = \text{CRNS}_{M*2^w}^{N*2^w}(r_j)$	% is Mont reduction
<i>DQ-RNS Mont [5]</i>	$a'_i = (((aM)\%p)2^{w/2}K)\%m_i$ $a'_j = (((aM)\%p)2^{w/2}K)\%n_j$	same as above	$q_j = \text{CRNS}_{N*1}^{M*d}(s_i)$	$r_i = \text{CRNS}_{M*1}^{N*K}(s_j + q_j)$ $r_j = (s_j + q_j)(\gamma_j)\%n_j$	$K = \text{ICRT}\left(\sqrt{c_i p^{-1}}\right)$ (if exists) $\gamma_j = 2^{3w/2}K \frac{N}{n_j} \% n_j$
<i>D-RNS Mont [6]</i>	$a'_i = (((aM)\%p)2^w)\%m_i$ $a'_j = (((aM)\%p)2^w)\%n_j$	same as above	$q_i = (s_i d'_i)\%m_i$ $q_j = \text{CRNS}_{N*2^{2w}}^{M*c^{-1}2^{-w}}(q_i)$	$r_j = (s_j + q_j)(W'_j)\%n_j$ $r_i = \text{CRNS}_{M*2^{2w}}^{N*c^{-1}2^{-w}}(r_j)$	$c_i = (\frac{M}{m_i})^{-1}\%m_i$ $d'_i = d_i c_i$ $W'_j = W_j c_j$
<i>Opt Mont</i>	$a'_i = (((aM)\%p)2^w)\%m_i$ $a'_j = (((aM)\%p)\textcolor{red}{M}^{-1}2^w)\%n_j$	same as above	$q_j = \text{CRNS}_{N*p\textcolor{red}{M}-2^{2w}}^{M*d2^{-w}}(s_i)$	$r_j = s_j + q_j$ $r_i = \text{CRNS}_{M*2^{2w}}^{N*\textcolor{red}{M}2^{-w}}(r_j)$	Diff with D-RNS Mont $d = -p^{-1}\%M$

TABLE 1. FORMULAS FOR VARIATIONS ON MODULAR MULTIPLICATION

CRNS_{N*z}^{M*y} is RNS conversion from RNS M to RNS N with premultiplication by y and postmultiplication by z . Computational changes from prior art to our optimized algorithm *Opt Mont* are highlighted in **red**. There are fewer multiplications and a simplified structure in relation to prior art.

multiple possible ways to represent the elements in this ring. Montgomery multiplication introduces another technique for fast arithmetic: working on a different representation. We can define this representation by its mapping from the standard representation. This mapping can be found in the representation column for the algorithm *Mont simple* in Table 1. As an example, the number 56 modulo 117 in Montgomery form with $w = 8$ would be represented by the number $(56)(256)\%117 = 62$. While this may seem counterintuitive at first, we are still able to perform all the computations correctly on this representation of the ring using Montgomery versions of multiplication and addition, and the choice of 256 allows us to use more efficient arithmetic than the traditional algorithm.

This representation is called Montgomery form. Because of the need to convert into and out of Montgomery form, Montgomery modular arithmetic is most useful for programs which perform a series of modular operations, such as that which occurs during modular exponentiation or repeated group operations, which occurs commonly in cryptography. It may be possible for the computation to stay entirely in Montgomery form, with parties communicating messages computed, stored, and verified in Montgomery form.

2.2. Redundant representations

A further technique to accelerating modular arithmetic is not only to use a different representation, but to use a redun-

dant representation. In a redundant representation, we allow multiple numbers (called representatives) for an element of the ring. For example, the number 56 modulo 117 can now be represented in redundant Montgomery representation by either the number 62 or the number 179. In fact, Table 1 for succinctness omits a key step for succinctness, called a conditional correction. The output r is actually a redundant representation with 2 representatives. The redundant representation can be converted to the standard representation by performing a comparison and conditional subtraction.

- if $(r \geq p)$, $r = r - p$

It is possible to skip this computation and compute on the redundant representation itself, provided that we are willing to store (and compute on) slightly larger numbers, which saves computational work. Using a redundancy of 2, each input to modular multiplication will be at most $2p$, with a product $s = a * b$ of at most $(2p)^2$ and quotient q of at most $2^2 p$.

Using redundant forms requires some slight modifications to related algorithms. Modular addition requires subtraction with the “redundant modulus” $2p$ instead of p :

- 1) $s = x + y$.
- 2) If $s \geq 2p$, $s = s - 2p$.

Modular multiplication requires that $2^w > q > (2^2)p$ rather than $2^w > q > p$. This mean we can use at most a $(w - 2)$ -bit modulus with w -bit Montgomery arithmetic (e.g., 62-bit

modulus with 64-bit arithmetic). It is possible to use even larger redundancy provided we do careful bookkeeping to ensure we do not overflow the datatype size (e.g., if we are storing in 64-bit numbers) [8]. In Appendix B, we will prove the correct bounds for a redundancy of 3, which is also required for our *Opt Mont* algorithm (last row of Table 1).

2.3. Residue Number System (RNS)

We take a brief interlude to discuss fast computation with composite (non-general) moduli. Specifically, moduli which can be factored into a product of word-size w (e.g., $w = 64$ on a 64-bit system) co-prime moduli. Consider a t -word modulus, meaning p is expressed with t words and $p < 2^{tw}$. In contrast to general modular arithmetic, which scales with big integer multiplication (usually quadratic $\Theta(t^2)$), these special moduli will have linear $\Theta(t)$ modular arithmetic algorithms.

This works by introducing yet another representation for numbers, called the Residue Number System (RNS) [9]. In RNS, each number is represented by its remainder in each factor of the modulus. Let these factors be $m_1, m_2 \dots m_t$, then we can do modular arithmetic mod $M = \prod_{i=1}^t m_i$ by performing operations in each small modulus independently. By choosing a fixed modulus size of w bits, this algorithm becomes linear time.

We will represent a number in RNS as a vector of modular residues.

$$\vec{a}_M = [a \% m_1, a \% m_2 \dots a \% m_t]$$

$$\vec{b}_M = [b \% m_1, b \% m_2 \dots b \% m_t]$$

For a number $a < M$ and $b < M$, we can compute $ab \% M$ by multiplying these two vectors with elementwise modular reduction $\odot$, meaning that each index of the vector can be processed in parallel.

$$\begin{aligned} \text{RNS}_M(ab) &= \text{RNS}_M(a) \odot \text{RNS}_M(b) \\ &= [ab \% m_1, ab \% m_2, \dots, ab \% m_t] \end{aligned}$$

2.4. RNS Montgomery modular multiplication

We will now discuss an algorithm usually only found on FPGAs, ASICs, and other dedicated hardware, called RNS Montgomery modular multiplication [1]. It works by using Redundant-Montgomery-Residue-Number-System (RM-RNS) to represent numbers and accelerate arithmetic. This algorithm has concretely more operations than Montgomery multiplication; there have been numerous optimizations to reduce the number of multiplications proposed. We will describe these optimizations below, referring to Table 1. In Section 3, we will further modify the representation to remove (we conjecture) all unnecessary elementwise modular multiplications (cf. Section 3.2) to arrive at our algorithm *Opt Mont*, briefly described in the last row of Table 1.

First, as a precursor, note the *Mont* and *Modular Mont* rows in Table 1 corresponding to a version of Montgomery

multiplication where, instead of doing division and multiplication with a hardware-friendly 2^w as in the *Mont simple* algorithm, we do division with some modulus M . By choosing an M that factors into co-prime word-size moduli (formally 2^w -smooth), we can use linear-time RNS arithmetic instead of hardware-accelerated 2^w arithmetic. Then, it suffices to use the RNS representation and modify the algorithm accordingly, as shown in the next row *RNS Mont* of Table 1. Efficiently combining the RNS and Montgomery representations requires the use of an auxiliary algorithm, CRNS, which allows a number to be switched between different RNS representations. In particular, we will switch between two (co-prime) RNS moduli: M and N in the algorithm. This switching will be the main computational step. The algorithm in the *RNS Mont* row is due to Posch and Posch [1]. Putting these steps together results in Algorithm 1.

Algorithm 1 RNS Mont Algorithm

```

1: procedure RNSMONTMUL( $\vec{a}_M, \vec{a}_N, \vec{b}_M, \vec{b}_N$ )
Require:  $\vec{a} = \text{RNS}(aM \% p)$   $\vec{d}_M = \text{RNS}_M(-p^{-1} \% M)$ ,
 $\vec{p}_N = \text{RNS}_N(p)$ ,  $\vec{M}_N^{-1} = \text{RNS}_N(M^{-1} \% N)$ 
2:  $\vec{q}_M = \vec{a}_M \odot \vec{b}_M \odot \vec{d}_M$ 
3:  $\vec{q}_N = \text{CRNS}_{M \rightarrow N}^{*1}(\vec{q}_M)$ 
4:  $\vec{c}_N = (\vec{a}_N \odot \vec{b}_N + \vec{q}_N \odot \vec{p}_N) \odot \vec{M}_N^{-1}$ 
5:  $\vec{c}_M = \text{CRNS}_{N \rightarrow M}^{*1}(\vec{c}_N)$ 
6: return  $\vec{c}_M, \vec{c}_N$ 
7: end procedure

```

A brief explanation of the steps, referring to the line numbers, is as follows:

- 2) The quotient q is computed as $-\frac{ab}{p}$ modulo M .
- 3) This quotient is converted to RNS_N from RNS_M for use in the next step.
- 4) The remainder $\frac{ab+qp}{M}$ is computed in RNS_N .
- 5) The remainder is converted back to RNS_M to function as input to the next multiplication operation.

2.5. CRNS with pre and post multiplication

There are several possible algorithms that can be used to implement CRNS [10] [11] [12]. We create a unified view incorporating optimizations from prior work to describe a highly parallel and efficient CRNS algorithm. In particular, $\text{CRNS}_{N \rightarrow M}^{*y}$ will perform 4 operations together:

- 1) Multiply by $y \pmod{M}$.
- 2) Apply the inverse Chinese Remainder Theorem (ICRT) to convert from RNS_M to a number.
- 3) Convert the number to RNS_N by computing the remainder $\% n_j$ for each moduli n_j .
- 4) Multiply by $z \pmod{N}$.

Step 1 and step 4 (pre- and post- multiplication, respectively) are added to the basic Change RNS modulus functionality of Posch and Posch [1]. These steps allow a unified view of the optimizations in prior work, as shown in Table 1.

Further optimization will lead to our final optimized algorithm *Opt Mont* (last row of Table 1) that we will describe in Section 3. Despite the complexity of this functionality, we show $\text{CRNS}_{N \times z}^{M \times y}$ can be implemented by mainly a matrix-vector multiplication with carefully-chosen constants (cf. Figure 1). The algorithm is divided into a precomputation and online phase, described in $\text{CRNSPrecomputation}$ and CRNS respectively in Algorithm 2. Note that when $y = z = 1$ this algorithm has the action of converting x from $\text{RMRNS } M$ to x' in $\text{RMRNS } N$.

We describe the various steps of Algorithm 2.

- 1) Each constant that will be multiplied by an element of $\vec{x}_M$ is multiplied by y . Then, multiplication by this constant will also multiply by y in addition to its original value.
- 2) The inverse Chinese Remainder Theorem states that the number $x = (\sum_{i=0}^t r_i \text{ICRT}_i) \% M$ for some precomputed constants ICRT_i . We can express this in vector form by replacing the $\%M$ with an indeterminate k multiple of M : $x = \vec{I}_M \cdot \vec{x}_M - kM$. We use a fixed-point approximation on line 4 with u bits of precision to set up the computation of k , as $k = \lfloor \frac{\sum_{i=0}^t r_i \text{ICRT}_i}{M} \rfloor$ in lines 10 and 11. This is an approximation in that the value computed may be off by one (cf. details in Appendix A).
- 3) The constants in the ICRT equation ($\vec{I}_M$ and M) are represented in the $\text{RNS } N$ by precomputing each value $\%n_j$ on lines 3 and 5.
- 4) Each constant in the ICRT equation is also multiplied by z , which means that multiplication with them also multiplies by z in addition to its original value.

The output of the algorithm will be in redundant form with a redundancy of 2 due to the error in the fixed-point approximation for k . The CRNS algorithm is used twice in RNS-Mont such that RM-RNS form has a total redundancy of 3, see Appendix B for details.

2.5.1. Example of Applying CRNS. We illustrate an example of CRNS in Figure 1 with $y = z = 1$. The input vector in mod M is used to compute both a fixed-point approximation for k , and to accumulate several precomputed values using vector instructions. The accumulated value is elementwise modular reduced in mod N , and it equals the input (but now in $\text{RNS } N$ rather than M) provided that the bounds in Appendix A are met.

2.5.2. Combining constants. Using the CRNS framework we can reduce the number of multiplications. In particular DQ-RNS Mont [5] combines the multiplication by $d = -p^{-1}$ into the conversion of the quotient q and D-RNS Mont [6] combines the multiplication by p with the conversion of the quotient q . We will use both of these optimizations our *Opt Mont* algorithm described in Section 3. Combining these optimizations reduces the number of elementwise modular multiplications required per residue from 5 to 3; we will further reduce this to 2.

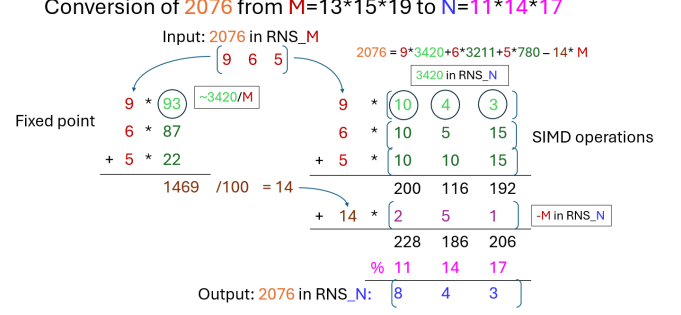


Figure 1. Example of change RNS base using vectors from $\text{RNS } M = 3705$ to $\text{RNS } N = 2618$. 3420, 3211, and 780 are the inverse Chinese remainder theorem factors for $\text{RNS } M$.

2.5.3. Double Montgomery form. We can further modify the representation by noting that each residue in RNS is itself a modular ring element. Therefore, we also can use Montgomery (or redundant Montgomery) form to represent the individual RNS residues, called double Montgomery form [6]. To use Montgomery form for the small residues, we need to convert to normal form before the CRNS operation and back to Montgomery afterwards. This can be done by including the Montgomery conversion factor 2^{2w} in the postmultiplication constant and the modular inverse 2^{-w} in the premultiplication constant.

3. Algorithmic optimizations and correctness

First, we describe our optimized RNS Montgomery multiplication algorithm *Opt Mont*, precomputation, and parameter requirements. We argue correctness in Section 3.2.

3.1. Full algorithm description

Our main modification is to precompute the multiplication by $W = M^{-1} \% N$ inside RNS_N . In particular, a number a in our new modified Montgomery form will be represented in two parts as $\vec{a}_M = \text{RNS}_M((aM) \% p)$ and $\vec{a}'_N = \text{RNS}_N(((aM) \% p) M^{-1} \% N)$, where the $\text{RNS } N$ component has been re-represented with a factor of M^{-1} , as highlighted in Table 1. We introduce the notation a' to refer to this modification. This algorithm is shown in Algorithm 3, excluding the double Montgomery optimization as it does not affect the number of multiplications. In Appendix B we show that the required bounds on the RNS are that $M > 9p$ and $N > 6p$.

3.2. Optimizing the number of multiplies and correctness argument

Instead of computing line 4 of Algorithm 1:

$$\vec{c}_N = (\vec{a}_N \odot \vec{b}_N + \vec{q}_N \odot \vec{p}_N) \odot \vec{M}^{-1}_N$$

We derive an expression by multiplying both sides of the equation by $\vec{M}^{-1}_N$, and distributing:

$$\vec{c}_N \odot \vec{M}^{-1}_N = \vec{a}_N \odot \vec{b}_N \odot \vec{M}^{-2}_N + \vec{q}_N \odot \vec{p}_N \odot \vec{M}^{-2}_N.$$

Algorithm 2 CRNS $_{N*z}^{M*y}$ subroutine

```

1: procedure CRNSPRECOMPUTATION( $M, y, N, z, u$ ) ▷ Do once
Require:  $M, y, N, z$  co-prime. ▷ Can be modified to handle not co-prime
2:  $\vec{I}_M \leftarrow I_i = ((\frac{My}{m_i})^{-1} \% m_i)(\frac{My}{m_i}) \% M$  ▷ Inverse Chinese Remainder Theorem with premultiplication
3:  $\mathbf{E} \leftarrow \mathbf{E}_{i,j} = z I_i \% n_j$  ▷ RNS with postmultiplication
4:  $\vec{f}^\top \leftarrow f_i^\top = \left\lceil \frac{I_i 2^u}{M} \right\rceil$  ▷ Ceiling fixed point with  $u$  bits of precision
5:  $\vec{g} \leftarrow g_j = (-zM) \% n_j$  ▷ ICRT Correction with postmultiplication
6: return  $\mathbf{E}, \vec{f}^\top, \vec{g}$ 
7: end procedure


---


8: procedure CRNS $_{N*z}^{M*y}(\vec{x}_M)$ 
Require:  $\vec{x}_M$  be  $x$  in RMRNS with modulus  $M, u \geq \lceil \log_2(\sum(m_i)) \rceil$ 
Ensure:  $\vec{x}'_N = (((x * y) \% M) * z) \% N$  in RMRNS with modulus  $N$ .
9:  $\mathbf{E}, \vec{f}^\top, \vec{g} = \text{CRNSPRECOMPUTATION}(M, y, N, z, u)$  ▷ Precomputed once
10:  $v = \vec{x}_M \cdot \vec{f}^\top$  ▷ Dot product; can be parallelized with  $\vec{x}_M \cdot \mathbf{E}$ 
11:  $k = \lfloor \frac{v}{2^u} \rfloor$  ▷ Bitshifting computes fixed-point quotient
12: return  $\vec{x}'_N = \vec{x}_M \cdot \mathbf{E} + k\vec{g}$  ▷ vector-matrix multiply, scalar-vector multiply, compute ICRT
13: end procedure

```

Algorithm 3 Opt Mont

```

1: procedure OPTMONT( $\vec{a}_M, \vec{a}'_N, \vec{b}_M, \vec{b}'_N$ )
2:  $\vec{s}_M = \vec{a}_M \odot \vec{b}_M \% m_i$ 
3:  $\vec{q}_N = \text{CRNS}_{N*pM-2}^{M*-p-1}(\vec{s}_M)$ 
4:  $\vec{r}'_N = (\vec{a}'_N \odot \vec{b}'_N + \vec{q}_N) \% n_j$ 
5:  $\vec{r}_M = \text{CRNS}_{M*1}^{N*M}(\vec{r}'_N)$ 
6: return  $\vec{r}_M, \vec{r}'_N$ 
7: end procedure

```

We then replace using the notation from Section 3.1:

$$\vec{c}_N \odot M^{-1}_N = \vec{c}'_N$$

$$\vec{a}_N \odot \vec{b}_N \odot M^{-2}_N = \vec{a}'_N \odot \vec{b}'_N.$$

Then, using the modified form it suffices to compute:

$$\vec{c}'_N = \vec{a}'_N \odot \vec{b}'_N + \vec{q}_N \odot \vec{p}_N \odot M^{-2}_N \quad (1)$$

The purpose of this notation is to illustrate how the new representation for the RNS $_N$ residues is $\vec{c}'$ rather than $\vec{c}$. The original value $\vec{c}_N$ can be recovered from $\vec{c}'$ by a multiplication with $\vec{M}_N$:

$$\vec{c}_N = \vec{c}'_N \odot \vec{M}_N.$$

This multiplication by M will be merged into the second CRNS (line 5 of Algorithm 3), which ensures that the inputs and outputs for the RNS $_M$ are unchanged. Since constants are merged together, this comes at no additional computational cost.

The constants pM_N^{-2} in Equation (1) are merged as the postmultiplication into the first CRNS, along with the constant $\vec{d}_M$ from line 2 (of Algorithm 1) as the premultiplication to make line 3 of Algorithm 3. These optimizations

greatly reduce the number of elementwise modular multiplications that have to be performed from 5 per residue to just the 2 required for computing $s' = a'b'$.

We will show in Section 5 that our implementation of this algorithm is almost fully vectorized.

4. Mapping to parallel hardware

4.1. Parallel commodity hardware

There have been many proposals for custom ASIC designs for RNS Montgomery multiplication. Unfortunately, these do not appear to have been built due to the large capital costs and scale required. There have also been FPGA design proposals, but these suffer from poor performance especially for large computational tasks. Realistic and practical deployment requires using existing acceleration hardware, perhaps originally designed for other tasks, to accelerate cryptography.

In this section we will describe how our RNS Montgomery algorithm maps to real hardware. We focus on the most challenging task of reducing latency for one instance of a modular multiplication or exponentiation, and would like to minimize the time this takes using whatever hardware resources are available. Throughput optimization using vectorization is relatively easy if there are independent multiplies, signatures, or other cryptographic tasks.

Each hardware accelerator will require different parameters to map the same mathematical description of the algorithm. A summary can be found in Table 2, and this section will describe the nuances required to create these implementations.

4.1.1. AVX instructions. AVX instructions perform single instruction multiple data (SIMD) functionality, applying the same operation to all elements of a vector. The size of this vector for AVX256 is 256 bits, and for AVX512 is 512

Implementation	Moduli size (w)	Accumulator size
CPU-AVX512	52 bits	116 (128) bits
GPU	32 bits	76 (96) bits

TABLE 2. COMPARISON OF IMPLEMENTATION PARAMETERS

Moduli size is equivalent to word size w , and accumulator must be at least $2w + \log_2(t)$ bits.

bits. Since AVX512 operations offer twice the throughput (operating on twice as many elements at once) of AVX256 instructions, we design our implementation for AVX512. These instructions have many variations and extensions which could be used to map the algorithm: we could use anything from 8-bit to 64-bit moduli and achieve vectorization. However, we find that the most efficient implementation is to use the AVX512-IFMA extension instruction set, which provides for 52-bit multiplication operations. This means that we will use 52-bit residues and have a 52-bit word size for our arithmetic. In particular, a 52-bit residue size reduces t by 38% compared to 32-bit residues (the maximum multiplication size for processors without AVX512-IFMA support). Furthermore, the 64-bit accumulator in AVX512-IFMA allows us to easily implement lazy reduction during CRNS. We provide a description of key AVX512 instruction in Table 3.

Mnemonic instruction	Description
_mm512_madd52lo_epu64	52-bit low multiplication with 64-bit accumulator
_mm512_madd52hi_epu64	52-bit high multiplication with 64-bit accumulator
_mm512_cmpge_epi64_mask	Compare greater than or equal, creating mask
_mm512_mask_sub_epi64	Subtract if mask is set

TABLE 3. SELECTED AVX512-IFMA AND RELATED INSTRUCTIONS

We do not consider multicore parallelization as the synchronization and data movement overheads are likely to be key bottlenecks.

4.1.2. GPU vectorization. Similar to a CPU, a GPU has multiple vector cores, and we will not consider multicore parallelization due to synchronization and data movement bottlenecks. GPUs use a SIMT architecture. As an example, we describe the layout for a RTX 3090 GPU, as described in the documentation³. The GPU executes computation in warps, groups of 32 threads. Each core has 4 schedulers, which means there can only ever be 4 warps active (scheduled at the same time), creating an effective parallelism of 128 threads per core. As each thread can execute one 32-bit multiplication operation, the behavior is equivalent to a SIMD AVX-4096.

4.2. Constructing an RNS

An RNS should consist of co-prime word-size moduli that are as large as possible. This can be done using a greedy

3. <https://docs.nvidia.com/cuda/cuda-c-programming-guide/#compute-capability-8-x>

algorithm, starting with $2^w - 1$, decrementing by 2, and checking if each new number is co-prime with respect to the current set of moduli, and including it when co-prime. We repeat until the total RNS modulus (the product of the selected moduli) MN exceeds $9p$ in the M and N parts, the bound required for the RNS Montgomery multiplication algorithm as derived in Appendix B. These numbers are found very quickly, as the probability of a w -bit number being prime is $1/w$ via the prime number theorem, and w is small; numbers are even more likely to be co-prime than prime. One consequence of this is that our RNS moduli will happen to be pseudo-Mersenne numbers, meaning that they are of the form $2^w - z$ for small z . This not only minimizes the number of RNS moduli t required to represent a number modulo p , but also allows additional optimizations in elementwise modular reductions.

4.2.1. Hardware-friendly moduli. ASIC papers [13] [14] use moduli with low Hamming weight, particular RNS moduli of the form $m_i = 2^w - z_i$ called pseudo-Mersenne numbers when z_i is small. These moduli admit an efficient hardware implementation using bitshifts and additions. This is because multiplications are fundamentally more expensive than additions, which are more expensive than bitshifts. However, on commodity hardware small pseudo-Mersenne moduli do not see specific speedups. Both CPUs [15] and GPUs [16] are designed to do 1 multiplication per “cycle” and cannot operate faster than this. Thus, integer multiplication, addition, and bitshift all take the same amount of time, despite the large differences in these operations on ASICs. In particular, pseudo-Mersenne moduli are not generally faster than general moduli, outside of the special cases of $2^w \pm 1$. These special cases are insufficient to build an RNS of the size required for cryptographic modular multiplication. We will instead show that the pseudo-Mersenne property can be taken advantage of in lazy reduction, which we call lazy partial pseudo-Mersenne reduction in Section 4.4.3.

4.2.2. Representing RNS with AVX. A number in RNS is represented by a vector of residues. For example, a 377-bit number will require $t = 8$ residues each with 52 bits, and each is stored in one of the eight 64-bit slots of a 512-bit vector. Larger moduli are held in multiple 512-bit vectors and padded as needed.

4.2.3. Representing RNS with GPU. A number in RNS is represented by a vector of residues, and these are distributed across the threads. Each thread holds one residue, and the associated data, i.e., the modulus, Montgomery factor, and precomputed constants. These threads are split across multiple warps when required. Interwarp shuffle instructions are used within a warp, and shared memory is used to communicate between warps.

4.3. Algorithm breakdown

Algorithm 4 presents Algorithm 3 computationally using three subalgorithms:

- 1) Elementwise modular multiplication.
- 2) CRNS.
- 3) Wide accumulator lazy reduction.

From the perspective of a vector element, or thread on a GPU, the computation will then be:

Algorithm 4 Vector Opt Mont

```

1: procedure OPTMONT( $\dot{a}, a', \dot{b}, b', m_i, n_i$ )
2:    $s = \text{MontReduce}(\dot{a}\dot{b}, m_i)$ 
3:    $q = \text{CRNS}_{N \times pM-2}^{M \times p-1}(s)$ 
4:    $r' = \text{WideMontReduce}(a'b' + q, n_i)$ 
5:    $\dot{r} = \text{WideMontReduce}(\text{CRNS}_{M \times 1}^{N \times M}(r'), m_i)$ 
6:   return  $\dot{r}, r'$ 
7: end procedure

```

We explain how to implement each subalgorithm using AVX in Section 4.4 and on a GPU in Section 4.5.

4.4. AVX

4.4.1. Implementing MontReduce. We use the simplified version of Montgomery (*Mont simple* in Table 1) for Montgomery reduction, instead of the standard version. Performing subtraction rather than addition allows us to truncate the computation of the numerator. Specifically, the ordinarily not true expression

$$\dot{c} = \frac{\dot{a}\dot{b} - qm_i}{2^w} = \left\lfloor \frac{\dot{a}\dot{b}}{2^w} \right\rfloor - \left\lfloor \frac{qm_i}{2^w} \right\rfloor$$

will be true in this setting. To see why, we can express the parts of the numerator in terms of quotients and remainders $\dot{a}\dot{b} = 2^w c_1 + r_1$ and $qm = 2^w c_2 + r_2$ where $0 \leq r_{\{0,1\}} < 2^w$. We know from the proof of correctness for Montgomery multiplication (cf. Appendix B) that the fraction is an integer, which implies the numerator is a multiple of 2^w . Then, we have that $(2^w c_1 + r_1) - (2^w c_2 + r_2) = 0\%2^w$, which implies that $(r_1 = r_2)\%2^w$. Because $0 \leq r_{\{0,1\}} < 2^w$, this means $r_1 = r_2$ (as integers).⁴ Since $r_1 = r_2$, then $\dot{c} = c_1 - c_2$. Finally, these are exactly the expressions given by truncation:

$$\left\lfloor \frac{\dot{a}\dot{b}}{2^w} \right\rfloor = c_1, \left\lfloor \frac{qm}{2^w} \right\rfloor = c_2$$

and therefore we obtain the correct answer using truncated results. For AVX instructions this reduces the required number of multiplications because high (maddhi) and low (maddlo) operations can be (and are) done separately. This is shown explicitly in Algorithm 5 with AVX vector instructions.

4. In contrast, traditional Montgomery gives $(r_1 = -r_2)\%2^w$, which ambiguously has multiple solutions as we defined r_1 and r_2 to be positive.

Algorithm 5 MontReduce with AVX

```

1: procedure MONTREDUCE( $\vec{h}\vec{i}, \vec{l}\vec{o}, \vec{m}$ )
Require: Input split into  $\vec{h}\vec{i}, \vec{l}\vec{o}$  words.  $\vec{D} = \vec{m}^{-1}\%2^{52}$ 
2:    $\vec{q} = \text{\_madd52lo\_epu64}(\vec{0}, \vec{l}\vec{o}, \vec{D})$ 
3:    $\vec{h} = \text{\_madd52hi\_epu64}(\vec{0}, \vec{q}, \vec{m})$ 
4:    $\vec{c} = \text{vec\_sub\_epi64}(\vec{h}\vec{i}, \vec{h})$ 
5:    $\text{mask} = \text{\_cmpgt\_epi64\_mask}(\vec{h}, \vec{h}\vec{i})$ 
6:   return  $\text{\_mask\_add\_epi64}(\vec{c}, \text{mask}, \vec{c}, \vec{m})$ 
7: end procedure

```

4.4.2. CRNS algorithm. The main operation of Algorithm 2 is a matrix-vector product. This operation technically happens over the RNS N , but it is computationally beneficial to delay the modular reduction step to after the matrix-vector product (so called lazy reduction [17]). All elements of the dot-product are accumulated before one wide reduction. This will necessitate the wide reduction algorithm (see Section 4.4.3), but results in a computational savings.

A key step in the matrix-vector product is the broadcasting of elements; this is performed with shared memory on GPU or AVX store/L1 cache on CPU. Each vector element, which we will call a thread to keep the terminology the same as in a GPU, can then accumulate the i th element of the matrix-vector product in parallel. Specifically, each thread computes the i th element of the output via a dot-product $\vec{x}_M \cdot \mathbf{E}_i$ to compute $\vec{x}_M \cdot \mathbf{E}$. This means that the values of $\vec{x}_M$ are broadcast to all threads, and each thread needs only 1 column of $\mathbf{E}$. We note that part of the fixed-point computation $v = \vec{x}_M \cdot \vec{f}^T$ has exactly the same form. Therefore, an additional thread can be used to accumulate v in parallel with the $\vec{x}_M \cdot \mathbf{E}_i$'s. After v is bitshifted to k , it can be broadcast back to the other threads in order to compute $k\vec{g}$. The accumulator size of 116 bits is given in the first row of Table 2. We require 116 bits because we are multiplying two 52-bit values and summing at most 2048 such products, and then adding $k\vec{g}$. Depending on how the AVX vectors pack values, the computation of v may be done separately using standard x86 instructions if the AVX vector would be mostly unused. This is the version illustrated in Algorithm 6 with the vector length t omitted for clarity, using the SIMD versions of instructions in Table 3 along with SIMD add, srli (bitshift right), and slli (bitshift left) pseudoinstructions. Each step of CRNS is mapped into several AVX instructions as follows:

- $v = \vec{x}_M \cdot \vec{f}^T$: Line 10 accumulates the fixed-point numerator v using x86 multiplication.
- $k = \lfloor \frac{v}{2^w} \rfloor$: Line 12 computes k using bitshifting.
- $\vec{x}_M \cdot \mathbf{E}$: Lines 7 – 9 using AVX 52-bit by 52-bit multiplication accumulated into a 116-bit accumulator *out*.
- $+k\vec{g}$: Lines 13 – 18 handle a 63-bit by 52-bit multiplication.

4.4.3. Implementing WideMontReduce. The lazy reduction in the previous part produces an accumulator larger than

Algorithm 6 CRNS using AVX instructions

```
1: procedure CRNS( $\vec{x}$ , CRNSPrecompute)
Require: Precomputed constants  $\mathbf{E}$ ,  $\vec{f}^\top$ ,  $\vec{g}$  as described in
    Algorithm 2.
2:   Store the AVX vector  $\vec{x}$  to memory in array  $\mathbf{y}$ 
3:    $\vec{out}_{lo} \leftarrow \vec{0}$ 
4:    $\vec{out}_{hi} \leftarrow \vec{0}$ 
5:    $v = 0$ 
6:   for  $i = 0 \dots t - 1$  do
7:      $\vec{y} \leftarrow \text{Broadcast}(\mathbf{y}[i])$ 
8:      $\vec{out}_{lo} = \text{\_madd52lo\_epu64}(\vec{out}_{lo}, \mathbf{E}_i, \vec{y})$ 
9:      $\vec{out}_{hi} = \text{\_madd52hi\_epu64}(\vec{out}_{hi}, \mathbf{E}_i, \vec{y})$ 
10:     $v = v + \vec{f}_i * \mathbf{y}[i]$   $\triangleright$  x86 mulx with 128-bit
        accumulator
11:   end for
12:    $k \leftarrow v \gg 64$   $\triangleright u = 64$  for fixed-point precision
13:    $\vec{k} \leftarrow \text{Broadcast}(k)$   $\triangleright k$  is 64 bits
14:    $\vec{out}_{lo} = \text{\_madd52lo\_epu64}(\vec{out}_{lo}, \vec{g}, \vec{k})$ 
15:    $\vec{out}_{hi} = \text{\_madd52hi\_epu64}(\vec{out}_{hi}, \vec{g}, \vec{k})$ 
16:    $\vec{out}_{hi} = \text{\_madd52lo\_...}(\vec{out}_{hi}, \vec{g}, \text{vec\_srli\_epi64}(\vec{k}, 52))$ 
17:    $\vec{tmp} = \text{\_madd52hi\_epu64}(\vec{0}, \vec{g}, \text{vec\_srli\_epi64}(\vec{k}, 52))$ 
18:    $\vec{out}_{hi} = \text{vec\_add\_epi64}(\vec{out}_{hi}, \text{vec\_slli\_epi64}(\vec{tmp}, 52))$ 
19:   return  $\vec{out}_{hi}, \vec{out}_{lo}$ 
20: end procedure
```

that which would normally be reduced by the Montgomery algorithm. Usually, Montgomery reduces $2w$ bits to w bits, but now the input can be up to $2w + \log_2(t)$ bits. We show that we can exploit the pseudo-Mersenne property to efficiently reduce the extra $\log_2(t)$ bits of the accumulator, allowing MontReduce to then be applied.

Let us consider the AVX-512 case concretely, where $w = 52$. First, we need to carry the upper 12 bits of the low accumulator to the high accumulator to normalize the 116-bit accumulator. Next, we reduce the extra 12 bits to create the $2 * 52 = 104$ -bit number expected in standard Montgomery. Consider the high 64 bits partitioned into its upper 12 bits u and lower 52 bits l . To perform partial reduction, observe that $(u2^{52} + l) \% (2^{52} - z) = u * z + l$, which can be computed using a single madd52lo operation. This is because u is at most 12 bits, and so for z of up to 40 bits, the product $u * z$ is at most 52 bits. Finally, we must follow up with a conditional correction because it is possible (however unlikely) that the sum $u * z + l$ could overflow to a 53-bit number. Afterwards, we will have an input that is valid for applying MontReduce from Section 4.4.1. This algorithm is shown in Algorithm 7.

4.4.4. Putting it together. Using the three subalgorithms we instantiate Algorithm 4 for AVX instructions in Algorithm 8. Negative exponents on constants should be understood as corresponding to precomputed modular inverses.

4.4.5. Bounds. We use 64-bit precision for fixed point, which allows up to $2^{11} = 2048$ residues while maintaining correctness (cf. Appendix A). This also ensures that the

Algorithm 7 WideMontReduce with AVX

```
1: procedure WIDEMONTREDUCE( $\vec{hi}, \vec{lo}, \vec{m}$ )
Require:  $\vec{mask} = 2^{52} - 1$ ,  $\vec{z} = 2^{52} \% \vec{m}$ 
2:    $\vec{hi} = \text{vec\_add\_epi64}(\vec{hi}, \text{vec\_srli\_epi64}(\vec{lo}, 52))$ 
3:    $\vec{lo} = \text{vec\_bitwise\_and}(\vec{lo}, \vec{mask})$ 
4:    $\vec{l} = \text{vec\_bitwise\_and}(\vec{hi}, \vec{mask})$ 
5:    $\vec{hi} = \text{\_madd52lo\_epu64}(\vec{l}, \text{vec\_srli\_epi64}(\vec{hi}, 52), \vec{z})$ 
6:    $\vec{mask} = \text{\_cmpge\_epi64\_mask}(\vec{hi}, \vec{m})$ 
7:    $\vec{hi} = \text{\_mask\_sub\_epi64}(\vec{hi}, \vec{mask}, \vec{hi}, \vec{m})$ 
8:   return  $\text{MONTREDUCE}(\vec{hi}, \vec{lo}, \vec{m})$ 
9: end procedure
```

Algorithm 8 OptMont with AVX

```
1: procedure RNSREDUCEAVX( $\vec{a}_M, \vec{a}_N, \vec{b}_M, \vec{b}_N$ )
Require: Co-prime modulus  $p$ , RNS  $M$  with moduli  $\vec{m}$  and
    RNS  $N$  with moduli  $\vec{n}$ , where inputs  $a$  and  $b$  are in the
    form described in Section 3.1
Ensure: Return  $ab \% p$  in the form described in Section 3.1
2:    $\vec{s}_{hi} = \text{\_madd52hi\_epu64}(\vec{0}, \vec{a}_M, \vec{b}_M)$ 
3:    $\vec{s}_{lo} = \text{\_madd52lo\_epu64}(\vec{0}, \vec{a}_M, \vec{b}_M)$ 
4:    $\vec{s} = \text{MONTREDUCE}(\vec{s}_{hi}, \vec{s}_{lo}, \vec{m})$ 
5:    $q_{hi}, q_{lo} = \text{CRNS}(\vec{s}, (-p^{-1})2^{-52}, (p)(M^{-2})2^{104})$ 
6:    $q_{hi} = \text{\_madd52hi\_epu64}(q_{hi}, \vec{a}_N, \vec{b}_N)$ 
7:    $q_{lo} = \text{\_madd52lo\_epu64}(q_{lo}, \vec{a}_N, \vec{b}_N)$ 
8:    $\vec{r}_N = \text{WIDEMONTREDUCE}(q_{hi}, q_{lo}, \vec{n})$ 
9:    $v_{hi}, v_{lo} = \text{CRNS}(\vec{r}_N, (M)2^{-52}, 2^{104})$ 
10:   $\vec{r}_M = \text{WIDEMONTREDUCE}(v_{hi}, v_{lo}, \vec{m})$ 
11:  return  $\vec{r}_M, \vec{r}_N$ 
12: end procedure
```

accumulators $\vec{out}_{hi}$ and $\vec{out}_{lo}$ in Algorithm 6 are at most $11 + 52 = 63$ bits, which leaves room for later additions without overflow. Therefore, the maximum number of RNS modulus bits in M or N is 106,496, and the supported modulus $p < M/9$ can have 106,492 bits.

4.5. GPU

We now describe mapping to a single GPU vector core. All steps in Algorithm 4 except for CRNS occur locally within a thread. MontReduce is implemented using Montgomery reduction as described in Table 1, and WideMontReduce is implemented in a similar manner to Section 4.4.3 but with $w = 32$. Using a 76-bit accumulator, as in Table 2, the uppermost 12 bits are reduced via multiplication with a $z < 20$ bits, and a conditional correction ensures the result fits as input to MontReduce.

We describe traditional parallelization for Montgomery, CRNS parallelization, and then our new technique of over-parallelization which fully exploits the abundant parallelism found in modern GPUs.

4.5.1. Parallelization of traditional algorithm. Many implementations implement Montgomery multiplication (*Mont* in Table 1), such as the CGBN [18] GPU library. The

algorithm will operate in a series of rounds, with each round reducing the unreduced product s by 1 word at a time, for a single modulus that is represented using t words. Each round has t parallel multiplications, along with 3 inter-thread broadcast operations. This repeats t times to reduce the unreduced $2t$ -word product to the t -word result. Then, an additional t rounds of inter-thread carry propagation and conditional correction are required to compute the Montgomery form result. Asymptotically, there are $\Theta(t^2)$ operations distributed across $\Theta(t)$ threads for a $\Theta(t)$ latency algorithm. CGBN does not use redundant representations (cf. Section 2.2) nor RNS Montgomery (cf. Section 2.4). A pictorial view is shown in Figure 9 in Appendix C.

4.5.2. CRNS parallelization. In Figure 2 we illustrate the data movement patterns for RNS Montgomery. `MontReduce` and `WideMontReduce` are thread-local and trivially parallelizable. The CRNS algorithm requires $t + 1$ rounds of broadcasts. In each round, an element of $\vec{x}_M$ is broadcast to $t + 2$ threads. t of these threads compute $\vec{x}_M \cdot E$ while two compute $\vec{x}_M \cdot \vec{f}^\top$. Then, the value of k is broadcast to the t threads, which accumulate $k\vec{g}$. In contrast to standard Montgomery, the data movement pattern consists only of broadcasts, and fewer broadcasts are required.

4.5.3. Overparallelization. We can do better by noting that with RNS Montgomery we do not have to parallelize to only t threads (the limit of standard Montgomery) with t moduli. We can in fact “overparallelize” by using more than t threads for the computation, achieving strong scaling. This is valuable when using computational hardware like a GPU where each vector core provides 128 threads.

The $\Theta(t)$ critical path length comes from the matrix-vector multiply step. The matrix-vector multiply is broken into additional threads by distributing each column dot-product among multiple threads and using addition reduction to combine the results back together. This is shown for one iteration in Figure 3. By using more threads the critical path latency is reduced: twice as many threads reduces the length (asymptotically) by $1/2$. This allows us to take full advantage of the parallelism of the GPU.

5. Performance Analysis

5.1. AVX speedup

An AVX512 instruction processes eight 64-bit chunks at once. However, to do 8 multiplies with 52-bit operands to produce 104-bit products⁵ by using a `maddlo` and `maddhi` operation requires 2 instructions, which effectively halves the number of simultaneous multiplies to 4. This means that we can expect the AVX512-IFMA throughput, in terms of multiplication, to increase by a factor of 4, which is consistent with our results in Figure 6.

5. The standard multiply would have 64-bit operands and produce a 128-bit product.

Step	Mults	Threads	Work
<code>MontReduce</code>	2	t	$2t$
CRNS: $\vec{x}_M \cdot E$ and $\vec{x}_M \cdot \vec{f}$	$2(t)$	$t + 2$	$2t^2 + 4t$
CRNS: k from v	$2(0)$	1	0
CRNS: $+k\vec{g}$	$2(1)$	t	$2t$
<code>WideMontReduce</code>	$2(3)$	t	$6t$
Algorithm 4 lines 2, 4	2	t	$2t$
Total	$2t + 12$		$2t^2 + 16t$

TABLE 4. BREAKDOWN OF PARALLELIZATION PER STEP FOR ALGORITHM 4 ON A GPU. MULTS IS MULTIPLICATIONS PER THREAD. 2() OPERATIONS OCCUR TWICE. TOTAL MULTS PER THREAD IS THE CRITICAL PATH LENGTH.

In addition, we use AVX instructions to accelerate other instructions in the algorithm (e.g., additions, shifts, etc.), not just multiplications, and so these will also benefit from AVX speedups. In particular, we use AVX512 instructions for everything in Algorithm 8, but that which occurs in lines 10 and 12 of Algorithm 6.

5.2. GPU speedup

A GPU vector core can parallelize over 128 threads creating a maximum speedup of 128 over a single thread. However, the GPU has some overheads if our baseline is a 64-bit CPU algorithm: 32-bit arithmetic requires twice as many residues for the same modulus bitsize as 64-bit arithmetic, resulting in $2^2 \times$ operations. In addition, GPUs can have slower frequencies and GPU kernels incur latency as data must be moved between GPU and CPU. GPUs are best used when many cryptographic operations will occur consecutively (like in modular exponentiation) that can take place wholly on the GPU.

5.3. Multiplication complexity

One method of measuring algorithmic complexity is to count the number of multiplication operations. The computational complexity of Montgomery multiplication (*Mont* in Table 1) given in terms of single precision multiplication operations is $2t^2 + 2t$ (Note 14.38 in [19]).

Using the values in Table 4, we can compute that the total work for Algorithm 4 is $2t^2 + 16t$, which is greater than the $2t^2 + 2t$ of the *Mont* algorithm. However, RNS Montgomery will have less parallelization overhead despite the increased work required. In the latency-focused setting, we need only consider the critical path. All of the work can be vectorized except for the $\Theta(1)$ computation of k from v (via bitshift). Using $t + 2$ threads on a vector core in the GPU produces a critical path of $2t + \Theta(1)$ as shown in the bottom row of Table 4.

By overparallelizing, we can reduce this even further (provided we have a machine with sufficient threads). Using $2t + 4$ threads produces a critical path of $t + \Theta(1)$. We can even produce a critical path length of $\Theta(\log(t))$; in particular, with $t^2 + 2t$ threads, each CRNS requires 1 multiplication per thread; an addition tree with $\log_2(t)$ additions finishes the computation of each dot product. Although

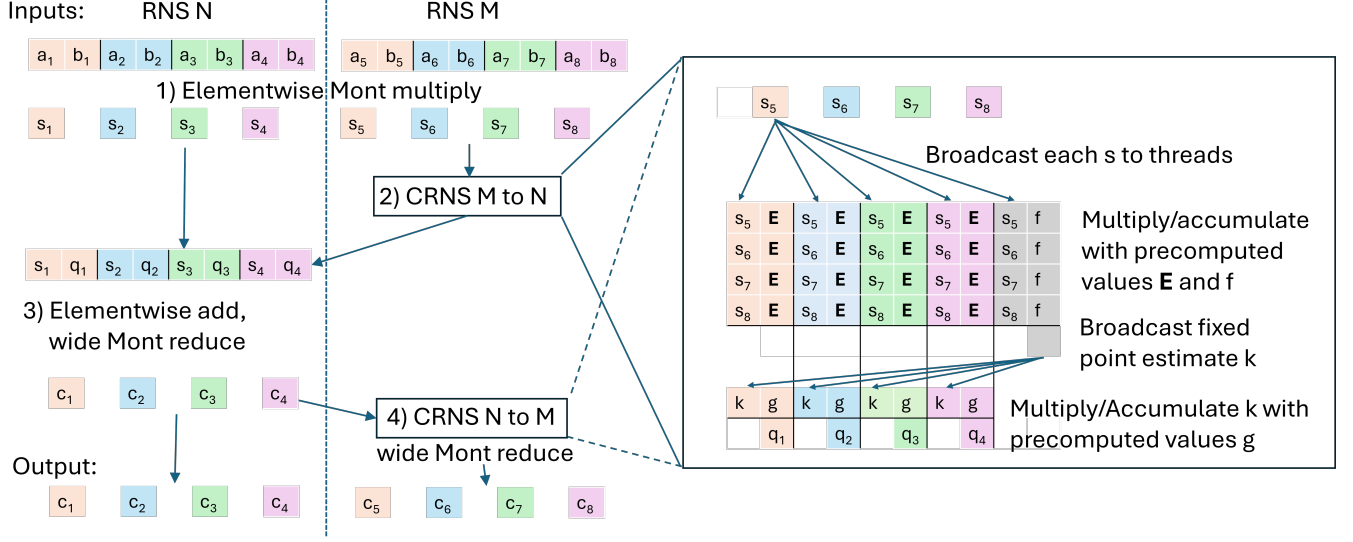


Figure 2. Illustration of the data movement pattern for optimized RNS Montgomery multiplication on a GPU. Steps 1, 2, 3, 4 correspond to Algorithm 4. Precomputed values E, f, g are as described in CRNSPrecompute (Algorithm 2). CRNS consists of broadcast rounds combined with local multiply-accumulates. Each of t threads computes $x \cdot E_i$ (line 12) while an auxiliary thread (gray) computes $v = x \cdot f$ (line 10), bitshifts (line 11), and broadcasts k . Finally the t threads accumulate $k\bar{g}$ (line 12).

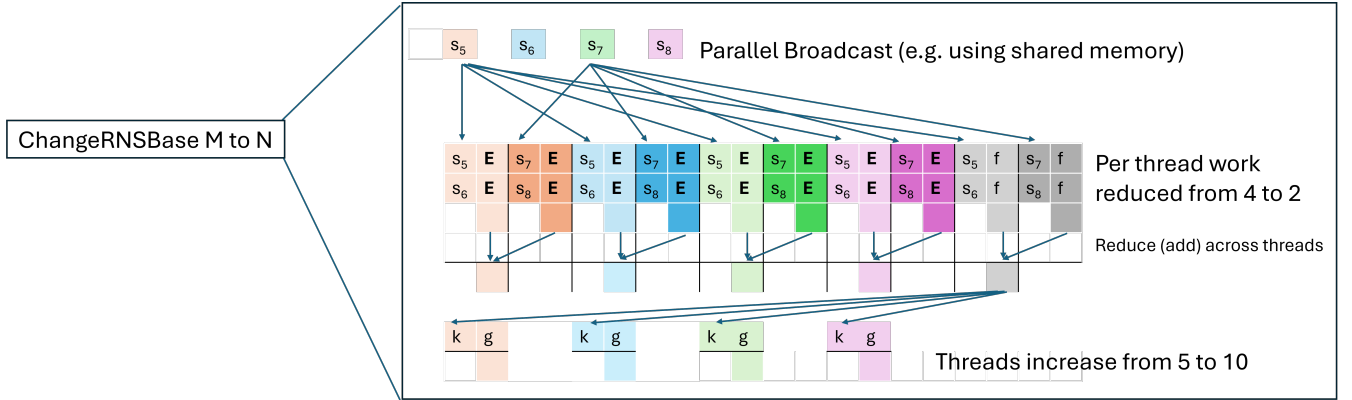


Figure 3. Using more than t threads reduces critical path length and provides strong scaling. Per thread work reduces with small parallelization overhead.

we can theoretically reduce the critical path from $\Theta(t)$ to $\Theta(\log(t))$, this is inefficient in practice on a GPU because of the inefficiency of inter-core communication required for full parallelization. We will restrict ourselves to using $2t+4$ threads in Section 6.

6. Evaluation

6.1. CPUs with AVX512

We implement Algorithm 8 on the SapphireRapids AWS `c7i.metal` –24xlarge machine on a single core. Our benchmark measures the time for a serial sequence of modular multiplications $a' = a * b \bmod p$ using our method or a library. This enables benchmarking of the latency (as each multiplication is dependent on the last).

We compare to two baseline implementations of modular multiplication: FLINT (Fast Library for Integer Number

Theory) [20] provides optimized modular arithmetic for mathematical and scientific applications. Flint uses two different datatypes/algorithms for primes up to 1024 bits and primes larger than 1024 bits. Arkworks [21] provides modular arithmetic for cryptographic algorithms and libraries in the Rust programming language. It is commonly used in elliptic curve based cryptography projects.

In Figure 4, we compare performance on moduli in 64-bit increments, so as to align with the baselines that store numbers as 64-bit limbs. In contrast, Algorithm 8 achieves its best efficiency every $52 \times 8 = 416$ bits, when an AVX512 vector is fully utilized. For 416 bits, we have $t = 8$.

To better illustrate the behavior for small moduli, we expand this region in Figure 5. As one might expect, we do not see speedups until the modulus size is approximately 512 bits in order to fully utilize AVX512 instructions. Following Table 3, it takes 2 AVX512 instructions to hide the

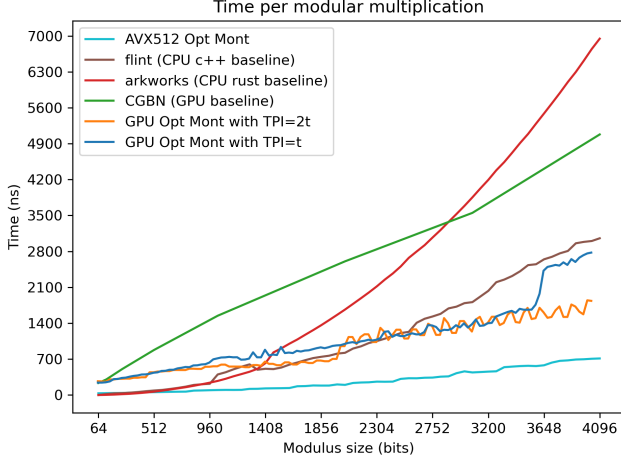


Figure 4. Comparison of AVX512, and GPU accelerated *Opt Mont* multiplication on a range of moduli of different bit sizes on a AWS c7i.metal-24xlarge (SapphireRapids) and RTX 3090. For more detail, see Figure 5 and Figure 8 for an enlargement of the small modulus region, and Figure 6 and Figure 7 for baseline comparisons. For an explanation of GPU vs CPU comparison, see Section 5.2.

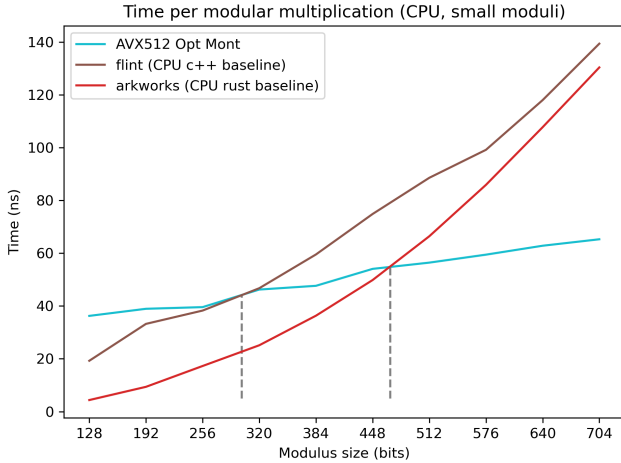


Figure 5. AVX512 *Opt Mont* multiplication becomes the fastest for 512+ bit moduli.

latency for multiplication, and so we cannot achieve full utilization of AVX compute units until we have at least 416 bits. AVX512 modular multiplication surpasses all baselines around 512 bits. For smaller moduli, it is possible to pack residues from multiple numbers into the same vector to fully utilize vectorization, but we leave this for future work. Even without this packing, RNS Montgomery is only a factor of 2 slower than Flint for small primes, e.g., 128 bits.

In Figure 6, we show the asymptotic speedup of the algorithm for large moduli over the baseline. The algorithm takes approximately 25% of the time as the best baseline, matching our expected speedup of $4\times$ from Section 5.

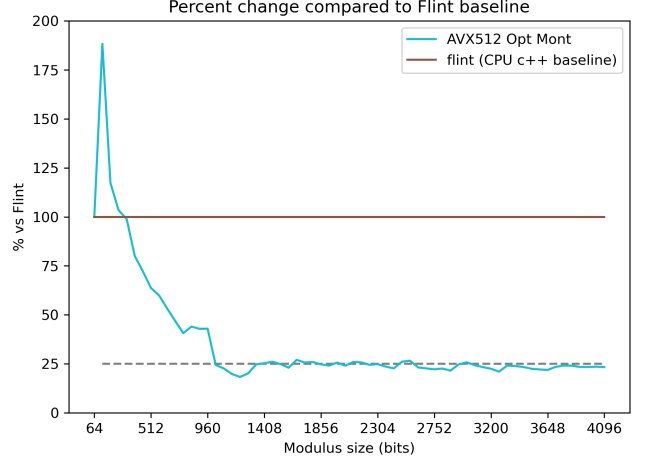


Figure 6. Percent change versus Flint baseline for large moduli. Speedups are approximately $4\times$ for 960+ bit moduli.

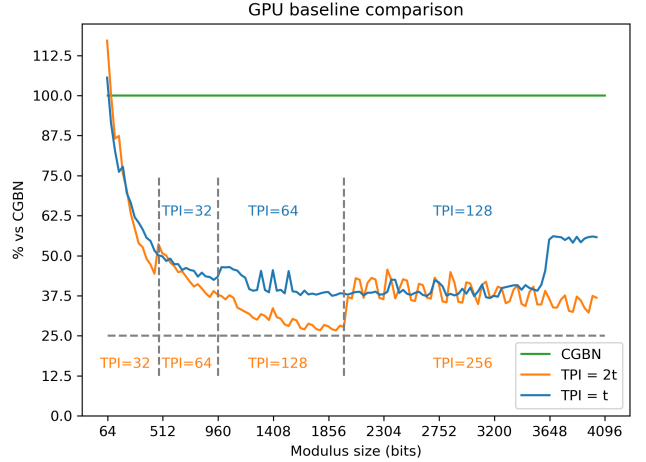


Figure 7. Comparison with CGBN GPU library (which supports up to 32 threads). Speedup approaches maximum $128/32 = 4\times$ thread ratio. $TPI = 2t$ uses all 128 threads for a 960+ bit modulus, and 256 threads for 1980+ bits, resulting in some scheduling overhead and performance comparable to 128 thread $TPI = t$. Beyond/at 3616 bits, $TPI = t$ requires spilling and performance decreases, while $TPI = 2t$ does not. Maximum modulus for *Opt Mont* shown is 4028 bits because of the 2 auxiliary threads. Warp shuffle instructions are used if they improve performance.

6.2. GPUs

We use an NVIDIA RTX 3090 for our GPU evaluation, and compare to the CGBN [18] bit integer and modular arithmetic library. CGBN uses a parameter “TPI” which stands for “threads per instance” – the number of threads used to perform a modular multiplication. We adopt this terminology. CGBN parallelizes the standard Montgomery for a t -word ($w = 32$) modulus among $TPI = t$ threads, for TPI up to 32. We evaluate two versions for our RNS Montgomery algorithm, the first with $TPI = t$ utilizes $t + 2$ threads to compute modular arithmetic. The second, $TPI = 2t$ utilizes overparallelization with a factor of 2, using

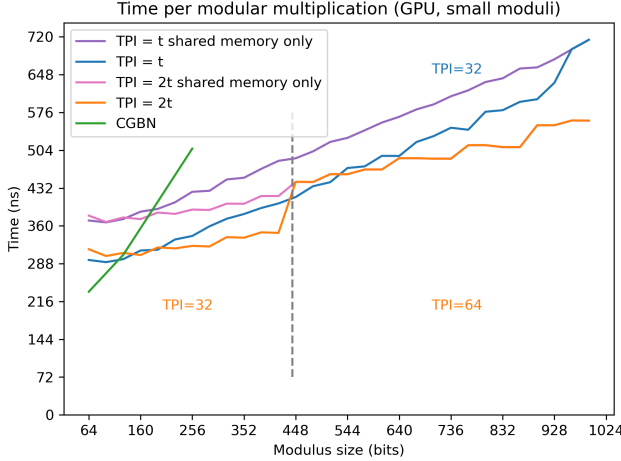


Figure 8. GPU based methods have significant starting latency. TPI = $2t$ improves performance for small moduli by utilizing more available threads. Warp shuffle instructions further improve performance when available (TPI ≤ 32); purple and pink lines indicate performance without.

$2t + 4$ threads to compute modular arithmetic. A comparison with CGBN can be found in Figure 4 and Figure 7 where we show that the speedup approaches $128/32 = 4\times$, the expected ratio for the number of threads used to compute in parallel. Figure 8 shows how using warp shuffle instructions improves performance, and the crossover point with CGBN.

Each thread can have 255 registers; therefore, the maximum number of registers with 128 threads is 2^{15} . However, we can actually have up to 2^{16} registers⁶, meaning that using 256 threads will allow more register utilization. Overparallelization thus allows the use of additional registers. Since running out of registers incurs expensive spilling, overparallelization allows supporting larger primes with more efficient scaling.

Exactly the same outputs are produced by all our algorithm variants and the three baselines, experimentally validating our implementation.

A final point worthy of note is that the multiplication algorithm used in modular multiplication libraries including Flint, Arkworks, and CGBN is the standard quadratic multiplication algorithm. For the size of primes in our experiments, asymptotically faster multiplication algorithms, e.g., Karatsuba [22], are not concretely more performant for modular reduction.

7. Additional Related work

There is a long history of algorithm development and optimizations for modular multiplication tailored for particular platforms. Our focus here is on important algorithm developments and optimizations to reduce the latency of a *single* modular multiplication and techniques that exploit

vectorization on commodity hardware, as opposed to ASICs or FPGAs.

As described in Section 2 and Table 1, our *Opt Mont* algorithm in Section 3 improves upon state-of-the-art, reducing the number of multiplications required. We show how to almost fully vectorize the algorithm for different hardware in Section 4.4 and Section 4.5.

Ochoa-Jimenez et al. [2] implemented Kawamura’s algorithm [5] on a CPU with AVX2 and achieved modest speedups for generating RSA signatures over prior interleaved [23] and multi-prime RSA [24] implementations. They reported a speedup for an application, as opposed to the modular multiplication primitive, through the use of parallelism as well as vectorization.

RNS Montgomery multiplication exploiting vectorization has been explored on GPUs (e.g., [3], [25]), ASICs (e.g., [26]), and FPGAs (e.g., [27], [17], [28], [14]). In [3], the focus is on parallelizing elliptic curve point multiplication, which is comprised of several independent field multiplications that can all be done in parallel on a GPU. Available GPU threads are assigned to different RNS moduli for different multiplications allowing for the use of a large number of threads. Significant speedup is reported, though it is not clear how much a single multiplication is sped up. [25] uses an older, less optimized RNS Montgomery algorithm and reports throughput results that are lower than conventional Montgomery on their GPU.

Finally, [29] describes a compiler for large integer arithmetic that outperforms CGBN by $1.42\times$ in terms of throughput for public-key cryptography, e.g., RSA. Conventional Montgomery multiplication is used.

8. Conclusion

Previously, RNS-based modular multiplication methods were largely limited to specialized ASIC and FPGA accelerators. Work on exploiting vector instructions has been fairly limited, and has not been deployed in libraries, to our knowledge. In this paper, we contributed a new RNS Montgomery algorithm with fewer multiplications and simpler structure than prior art, and showed that speedups are possible via vectorization, when prime sizes are larger than 256 bits. The RNS-based method becomes four-fold faster than traditional methods for primes over 1024 bits on CPUs with AVX512 extensions. Similar speedups are obtained on a GPU vector core in comparison to a commercial library.

Ongoing work includes implementing the extensions of Appendix D and exploring speedups for elliptic curve cryptography, and an RNS-based modular multiplication algorithm that exploits tensor cores on a GPU, in addition to vectorization.

6. <https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html#features-and-technical-specifications>

References

- [1] K. C. Posch and R. Posch, “Modulo reduction in residue number systems,” *IEEE Transactions on Parallel and Distributed Systems*, vol. 6, no. 5, pp. 449–454, 1995.
- [2] E. Ochoa-Jimenez, L. Rivera-Zamarripa, N. Cruz-Cortes, and F. Rodriguez-Henriquez, “Implementation of rsa signatures on gpu and cpu architectures,” *IEEE Access*, vol. 8, pp. 1–1, 01 2020.
- [3] S. Antão, J.-C. Bajard, and L. Sousa, “Rns-based elliptic curve point multiplication for massive parallel architectures,” *The Computer Journal*, vol. 55, no. 5, pp. 629–647, 2012.
- [4] P. L. Montgomery, “Modular multiplication without trial division,” *Mathematics of computation*, vol. 44, no. 170, pp. 519–521, 1985.
- [5] S. Kawamura, Y. Komano, H. Shimizu, and T. Yonemura, “Rns montgomery reduction algorithms using quadratic residuosity,” *Journal of Cryptographic Engineering*, vol. 9, 11 2019.
- [6] J.-C. Bajard and N. Merkiche, “Double level montgomery cox-rower architecture, new bounds,” in *Smart Card Research and Advanced Applications*, M. Joye and A. Moradi, Eds. Cham: Springer International Publishing, 2015, pp. 139–153.
- [7] P. Barrett, “Implementing the rivest shamir and adleman public key encryption algorithm on a standard digital signal processor,” in *Advances in Cryptology — CRYPTO ’86*, ser. Lecture Notes in Computer Science, A. M. Odlyzko, Ed., vol. 263. Springer, 1987, pp. 311–323.
- [8] D. Harvey, “Faster arithmetic for number-theoretic transforms,” *Journal of Symbolic Computation*, vol. 60, pp. 113–119, 2014.
- [9] N. S. Szabo and R. I. Tanaka, *Residue Arithmetic and its Applications to Computer Technology*. New York: McGraw-Hill, 1967.
- [10] A. Shenoy and R. Kumaresan, “Fast base extension using a redundant modulus in rns,” *IEEE Transactions on Computers*, vol. 38, no. 2, pp. 292–297, 2002.
- [11] J.-C. Bajard and T. Plantard, “Rns bases and conversions,” in *Advanced Signal Processing Algorithms, Architectures, and Implementations XIV*, vol. 5559. SPIE, 2004, pp. 60–69.
- [12] S. Kawamura, M. Koike, F. Sano, and A. Shimbo, “Cox-rower architecture for fast parallel montgomery multiplication,” in *Advances in Cryptology—EUROCRYPT 2000: International Conference on the Theory and Application of Cryptographic Techniques Bruges, Belgium, May 14–18, 2000 Proceedings 19*. Springer, 2000, pp. 523–538.
- [13] J.-C. Bajard, M. Kaihara, and T. Plantard, “Selected rns bases for modular multiplication,” in *2009 19th IEEE Symposium on Computer Arithmetic*. IEEE, 2009, pp. 25–32.
- [14] T. Wu, “High-performance rns modular exponentiation by sum-residue reduction,” *IEEE Canadian Journal of Electrical and Computer Engineering*, vol. 46, no. 2, pp. 137–143, 2023.
- [15] A. Abel and J. Reineke, “uops.info: Characterizing latency, throughput, and port usage of instructions on intel microarchitectures,” in *ASPLOS*, ser. ASPLOS ’19. New York, NY, USA: ACM, 2019, pp. 673–686. [Online]. Available: <http://doi.acm.org/10.1145/3297858.3304062>
- [16] NVIDIA, “5.4.1 arithmetic instructions: Cuda c++ programming guide,” <https://docs.nvidia.com/cuda/cuda-c-programming-guide/#arithmetic-instructions>, 2025, accessed: 2025-04-07.
- [17] R. C. Cheung, S. Duquesne, J. Fan, N. Guillermin, I. Verbauwheide, and G. X. Yao, “Fpga implementation of pairings using residue number system and lazy reduction,” in *International Workshop on Cryptographic Hardware and Embedded Systems*. Springer, 2011, pp. 421–441.
- [18] N. Emmart and J. Luitjens, “Cgbn: Cuda accelerated multiple precision arithmetic (big num) using cooperative groups,” 2019. [Online]. Available: <https://github.com/NVlabs/CGBN>
- [19] P. v. O. A. Menezes and S. Vanstone, *Handbook for Applied Cryptography*. CRC Press, 1996.
- [20] T. F. team, *FLINT: Fast Library for Number Theory*, 2025, version 3.2.1, <https://flintlib.org>.
- [21] arkworks contributors, “arkworks zksnark ecosystem,” 2022. [Online]. Available: <https://arkworks.rs>
- [22] A. A. Karatsuba and Y. Ofman, “Multiplication of multidigit numbers on automata,” *Soviet Physics Doklady*, vol. 7, pp. 595–596, 1962.
- [23] J. W. Bos, P. L. Montgomery, D. Shumow, and G. M. Zaverucha, “Montgomery multiplication using vector instructions,” in *Selected Areas in Cryptography – SAC 2013*, T. Lange, K. Lauter, and P. Liasoněk, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2014, pp. 471–489.
- [24] S. Gueron and V. Krasnov, “Speed records for multi-prime rsa using avx2 architectures,” in *Information Technology: New Generations*, S. Latifi, Ed. Cham: Springer International Publishing, 2016, pp. 237–245.
- [25] R. Szerwinski and T. Güneysu, “Exploiting the power of gpus for asymmetric cryptography,” in *International Workshop on Cryptographic hardware and embedded systems*. Springer, 2008, pp. 79–99.
- [26] K. Bigou and A. Tisserand, “Rns modular multiplication through reduced base extensions,” in *2014 IEEE 25th International Conference on Application-Specific Systems, Architectures and Processors*. IEEE, 2014, pp. 57–62.
- [27] —, “Improving modular inversion in rns using the plus-minus method,” in *Cryptographic Hardware and Embedded Systems-CHES 2013: 15th International Workshop, Santa Barbara, CA, USA, August 20-23, 2013. Proceedings 15*. Springer, 2013, pp. 233–249.
- [28] S. Duquesne and N. Guillermin, “A fpga pairing implementation using the residue number system,” *Cryptology EPrint Archive*, 2011.
- [29] Z. Ji, J. Zhao, Z. Zhang, J. Xu, S. Yan, and L. Ju, “A compiler-like framework for optimizing cryptographic big integer multiplication on gpus,” in *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2024, pp. 380–392.

Appendix A. CRNS error analysis

We prove correctness bounds for the CRNS algorithm. In this section we show correctness for $y = z = 1$, and in the next for other constants. For RNS word-size moduli of at most 2^w and a fixed-point precision of $u \geq w + \log(t) + 1$, the function CRNS_{N*1}^{M*1} has the following behavior:

$$\text{CRNS}_{N*1}^{M*1}(x) = \begin{cases} x \% N & x \leq M/2 \\ x(+)M \% N & x > M/2 \end{cases}$$

Proof: We decompose each fixed-point approximation into its real value and an error term of at most 1, created by the ceiling operator. Let ICRT_i be the precomputed constants from the inverse Chinese Remainder Theorem such that $x = (\sum_{i=1}^t r_i \text{ICRT}_i) - kM$. Let the ceiling rounded u bit fixed point approximation be

$$F_i = \left\lceil \frac{\text{ICRT}_i 2^u}{M} \right\rceil = \frac{\text{ICRT}_i 2^u}{M} + e_i, 0 \leq e_i < 1$$

Then, the evaluated expression is

$$\tilde{k} = \left\lfloor \frac{\sum_{i=1}^t r_i F_i}{2^u} \right\rfloor = \left\lfloor \frac{\sum_{i=1}^t r_i \frac{\text{ICRT}_i 2^u}{M} + \sum_{i=1}^t r_i e_i}{2^u} \right\rfloor$$

which simplifies to

$$\tilde{k} = \left\lfloor \frac{\sum_{i=1}^t r_i \text{ICRT}_i}{M} + \frac{\sum_{i=1}^t r_i e_i}{2^u} \right\rfloor$$

We can replace $\frac{\sum_{i=1}^t r_i \text{ICRT}_i}{M}$ by $k + \frac{x}{M}$ to get:

$$\tilde{k} = k + \left\lfloor \frac{x}{M} + \frac{\sum_{i=1}^t r_i e_i}{2^u} \right\rfloor$$

We now consider our two cases. For $x < M/2$ we have:

$$\left\lfloor \frac{x}{M} + \frac{\sum_{i=1}^t r_i e_i}{2^u} \right\rfloor < \left\lfloor \frac{1}{2} + \frac{\sum_{i=1}^t r_i e_i}{2^u} \right\rfloor$$

Therefore, for $\frac{\sum_{i=1}^t r_i e_i}{2^u} < \frac{1}{2}$, this term will be 0. Note that $r_i < m_i \leq 2^w$ for word size moduli, so we have:

$$\frac{\sum_{i=1}^t r_i e_i}{2^u} < \frac{\sum_{i=1}^t 2^w e_i}{2^u} < \frac{\sum_{i=1}^t 2^w}{2^u} = \frac{t2^w}{2^u}$$

Then, for $u > w + \log(t) + 1$, this is less than $1/2$, and we output $\tilde{k} = k$. For the case where $x > M/2$, we note that we will have by a similar argument:

$$\frac{x}{M} + \frac{\sum_{i=1}^t r_i e_i}{2^u} < \frac{3}{2}$$

After taking the floor, we have $\tilde{k} = k(+1)$. It follows from the full algorithm that this will turn into an output of $(k+1)M = kM + M$, and therefore output $x + M$. Applying this formula to Algorithm 6, we determine that $64 \geq 52 + \log(2048) + 1$ will be sufficient precision for $t < 2048$.

A.1. Constant Merging

CRNS_{N*z}^{M*y} is equivalent to the functionality illustrated in Algorithm 9.

Algorithm 9 CRNS_{N*z}^{M*y} Functionality

- 1: **procedure** $\text{CRNS}(\vec{x}_M, y, z)$
 - 2: $\vec{x}'_M = \vec{x}_M \odot \text{RNS}_M(y)$
 - 3: $\vec{x}'_N = \text{CRNS}_{N*z}^{M*1}(\vec{x}'_M)$
 - 4: **return** $\vec{x}'_N \odot \text{RNS}_N(z)$
 - 5: **end procedure**
-

First, the effect of post-multiplication can be seen by applying the distributive property to line 4 of Algorithm 2, namely that

$$\begin{aligned} \vec{x}_N \odot \text{RNS}_N(z) &= \sum_{i=1}^t r_i \vec{I}_{iN} \odot \text{RNS}_N(z) \\ &\quad + k \vec{M}_N \odot \text{RNS}_N(z) \end{aligned}$$

The elementwise multiplication operations $\odot$ between the constants $\vec{I}_{iN}$ and $\text{RNS}_N(z)$, and between $\vec{M}_N$ and $\text{RNS}_N(z)$ are precomputed.

The effect of pre-multiplication is similar but on the input. Observe that

$$\left(\sum_{i=1}^t (x \% m_i) ((\text{ICRT}_i * y) \% M) \right) \% M = (x * y) \% M$$

Therefore, evaluating this modified ICRT theorem outputs $\vec{x}_M \odot \text{RNS}_M(y)$.

Appendix B. Redundance Bound

For $M > 9p$ and $N > 6p$, Algorithm 1 will, for redundant Montgomery inputs of at most $3p$, output an redundant Montgomery representation of $c = ab \% p$ of at most $3p$.

A standard proof for Montgomery multiplication is as follows:

- 1) The calculation $\frac{c}{R}$ outputs an integer. This is because:

$$\begin{aligned} c \% R &= (\dot{a}\dot{b} + ((\dot{a}\dot{b}(-p^{-1} \% R))p)) \% R \\ &= (\dot{a}\dot{b} - \dot{a}\dot{b}) \% R \\ &= 0 \end{aligned}$$

- 2) The integer is equal to $(abR \% p)$. This is because:

$$\frac{(aR \% p)(bR \% p) + qp \% p}{R} \% p = \frac{abR^2 + 0}{R} \% p = abR \% p.$$

- 3) The integer has value at most $2p$, and therefore is a redundant representation. This is because $q < R$ and therefore:

$$\frac{\dot{a}\dot{b} + qp}{R} < \frac{\dot{a}\dot{b}}{R} + p$$

Specifically, for $\dot{a} < 3p$, $\dot{b} < 3p$, and $R = M > 9p$ we will have:

$$\frac{\dot{a}\dot{b}}{R} + p < \frac{(3p)(3p)}{9p} + p < 2p.$$

The only difference that occurs in RNS is that we may have $(\dot{a}\dot{b}d) \% M > M/2$. ChangeRNSBase will potentially output $x + M$ (Appendix A). Following the effects through the algorithm we have

$$\frac{ab + (q + M)p}{M} = \frac{ab + qp}{M} + p < 2p + p$$

which therefore outputs a redundancy for $\dot{c}$ of $3p$ instead of $2p$ in the standard (non-RNS) algorithm. To finish correctness, note that $\dot{c} < 3p < \frac{N}{2}$ for $N > 6p$, so by Appendix A, the second ChangeRNSBase will output $\dot{c}$ exactly in RNS_M .

Appendix C. CGBN Parallelization

Figure 9 shows how CGBN parallelizes the conventional Montgomery algorithm on a GPU.

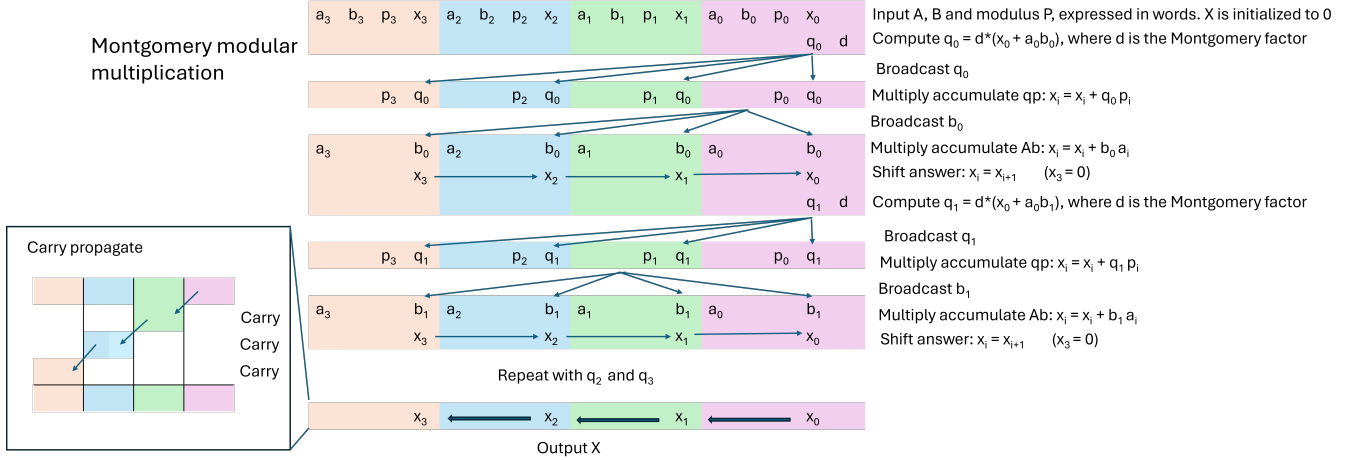


Figure 9. Parallelization of traditional Montgomery arithmetic algorithm as in CGBN. Final steps of carry propagation and conditional correction greatly increase critical path length.

Appendix D. Extensions

In this section, we provide an overview of several algorithms that are helpful for implementing other functions used by cryptographic applications.

D.1. Conversion to/from double RMRNS

We can use a modification of Algorithm 2 to efficiently convert inputs into RNS Montgomery form. The only difference is that inputs are expressed in terms of words rather than residues. Let $\vec{x} \leftarrow r_i : x = \sum_{i=t} r_i 2^{wi}$ instead of $r_i = x \% m_i$. Then, to convert x into double RMRNS form we can compute:

$$\text{CRNS}_{N \cdot M - 1 \cdot 2^{2k}}^{p \cdot M}(x)$$

This calculates RNS_N , and the expansion step (Algorithm 3 line 5) can calculate RNS_M . This means that conversion is almost equivalent, in computational cost, to a single modular multiplication.

A similar algorithm can be derived for conversion back to radix, however it will require some big integer steps to handle the output. This conversion will work with either RNS_M or RNS_N , but RNS_N allows omission of the expansion step (Algorithm 3 line 5) in the final multiplication:

$$\text{CRNS}_{p \cdot M - 1}^{N \cdot M \cdot 2^{-k}}(x)$$

For an efficient implementation, this algorithm accumulates on 64 bit accumulators before recombining the bits into the output big integer. A final step is to apply conditional correction to remove the redundancy.

D.2. Addition

Addition is implemented elementwise in RNS form. However, because we are using redundant RNS form, the

redundance grows after each step. After one addition, we would have $x + y < 6p$, rather than $3p$. For certain algorithms, such as elliptic curves, where the number of additions between multiplications is bounded, it is possible to compute a maximum required redundancy and expand the RNS accordingly. Otherwise, we can correct redundancy via modular multiplication with 1 at amortized intervals: accumulate up to the RNS size of $9p \cdot 6p = 54p^2$, and then perform modular reduction (without multiplication).

D.3. Subtraction

Traditional modular subtraction can be performed by the identity $a - b = a + (p - b)$. The equivalent in redundant RNS is to do $\text{RNS}(a) + \text{RNS}(3p) - \text{RNS}(b)$, where $\text{RNS}(3p)$ is precomputed, and the 3 is the bound on the redundancy. The same techniques as in addition apply to maintain a non-overflowing redundant form.

D.4. Equality

We can expand the range of supported programs from just circuits by introducing a fast modular comparison operator. This is important for some elliptic curve algorithms with special cases, and for performing general non-modular arithmetic using RNS form.

First, we note that $x == y \% p$ if and only if $(x - y) == 0 \% p$. It suffices to show an algorithm to efficiently test if a RNS (modulo M) represents a number that is 0 (modulo p). Observe that the output $x \in [0, 3p)$ can be expressed as $r + lp$, with $0 \leq r < p$ and $0 \leq l < 3$. Therefore, if $x == 0 \% p$, we will have $x(p^{-1}) \% M = (0 + lp)(p^{-1}) \% M = l \% M$. Now, note that for $l < 3$, $l \% M$ expressed in RNS form is $l, l, l, \dots, l$. To summarize: to test if x is the RNS form of 0 modulo p , we check if all of the residues are equal, and at most 3. A proof follows, showing how to also correct for the Montgomery form.

D.4.1. Equality check correctness. To show that this is sufficient, consider $x = r + lp$, $r > 0$. Then, $x(p^{-1})\%M = r(p^{-1})\%M + l$. It follows that for $r \geq 1$, $r(p^{-1})\%M + l > B$. This is because the multiplication by (p^{-1}) is a one-to-one mapping, and therefore other x cannot be mapped into $0 \dots l$. Furthermore, for most values of p , we have that $(p^{-1}\%M) > m_i$ for some m_i . In this case, it follows that $r(p^{-1})\%M + l > m_i$ and at least one of the residues will not equal the others, and the comparison to B is unnecessary. This is because the only numbers that have all residues equal are precisely those in the range $[0, \min m_i)$, as RNS form is a one-to-one mapping. To remove the elementwise Montgomery factor of 2^{52} , we simply multiply by the (pre-computed) combined constant $2^{-52}p^{-1}\%M$.